

Sistema de transacciones financieras por voz en dispositivos móviles iOS

Ing. Alexis Geraldine Barniquez Piñero

Carrera de Especialización en Inteligencia Artificial

Director: Ing. Sergio Ivaldi

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, junio de 2026

Resumen

En la presente memoria se desarrolla una prueba de concepto de una aplicación móvil nativa para iOS, capaz de procesar comandos de voz e interpretar intenciones transaccionales de forma 100 % local. Este sistema fue desarrollado para ser validado por el equipo de Identidad Digital del Banco Galicia Minorista.

Para su materialización, se utilizaron herramientas del ecosistema Apple como Swift, Xcode, la API AVFoundation y el framework Core ML.

Índice general

Resumen	I
1. Introducción general	1
1.1. Contexto de la banca móvil	1
1.2. Estado del arte	2
1.3. Objetivos y alcance	3
1.3.1. Alcance y limitaciones	3
1.4. Motivación: accesibilidad e inclusión	4
2. Introducción específica	5
2.1. Marco metodológico	5
2.2. Herramientas utilizadas	5
2.3. Requerimientos y modelos de IA utilizados	7
2.4. Exploración teórica de arquitecturas evaluadas	7
3. Implementación y desarrollo	9
3.1. Arquitectura del sistema	9
3.2. Tratamiento y preparación de datos	13
3.2.1. Creación de plantillas de comandos y aumentación de datos	13
3.2.2. Modelo de datos	13
3.3. Desarrollo del SDK (Aurum)	14
3.4. Desarrollo de la aplicación base	15
Bibliografía	19

Índice de figuras

3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.	11
3.2. Flujo de pipeline de una transacción realizada por voz.	12
3.3. Diagrama de flujo de Aurum.	16

Índice de tablas

Capítulo 1

Introducción general

El presente capítulo enmarca el desarrollo de una solución basada en interfaces conversacionales impulsadas por inteligencia artificial. A lo largo de las siguientes secciones se analizará el estado del arte de estas tecnologías, justificando la necesidad crítica de implementar un procesamiento local de los datos para garantizar la seguridad de la información financiera, y se define de manera precisa los objetivos y el alcance metodológico de la prueba de concepto a desarrollar en el ecosistema iOS

1.1. Contexto de la banca móvil

En la última década, la banca digital ha experimentado una transformación radical [1], al migrar de plataformas web tradicionales a aplicaciones móviles que se han consolidado como el principal punto de contacto entre los clientes y las instituciones financieras. En este entorno de alta competitividad, la innovación continua en la experiencia de usuario se ha vuelto un diferenciador estratégico clave para la retención y satisfacción del cliente. Las interfaces conversacionales, impulsadas por los recientes avances en inteligencia artificial y procesamiento de lenguaje natural, representan la siguiente frontera en la interacción humano-computadora, lo que permite una comunicación más fluida, rápida e intuitiva [2]. Este trabajo se enmarca en dicha evolución tecnológica, explorando la viabilidad de aplicar tecnologías de reconocimiento y comprensión de voz para la ejecución de operaciones financieras directamente desde el ecosistema de una aplicación de banca móvil.

En el contexto argentino en particular, la adopción de la banca móvil se aceleró de manera significativa a partir de la pandemia de COVID-19, que en 2020 forzó a las instituciones financieras a consolidar sus canales digitales como alternativa al servicio presencial [3]. Según datos del Banco Central de la República Argentina (BCRA), el número de transferencias inmediatas a través de plataformas digitales creció sostenidamente en los años subsiguientes, y el segmento de clientes que gestiona sus finanzas exclusivamente a través del canal móvil representa hoy una fracción sustancial de la base de usuarios de los principales bancos privados del país [4]. El Banco Galicia [5], como una de las entidades financieras privadas de mayor envergadura del país, no es ajeno a esta tendencia: sus plataformas digitales concentran la gran mayoría de las interacciones cotidianas de sus clientes, como consultas de saldo, transferencias, pagos de servicios y operaciones de inversión.

Es en este escenario de alta digitalización y creciente demanda de experiencias de usuario personalizadas donde la interfaz de voz emerge como una propuesta de valor diferencial. La interacción por voz reduce la carga cognitiva asociada a la navegación entre menús y formularios, y permite ejecutar operaciones en contextos donde el uso de las manos no es posible o no resulta conveniente. Esta convergencia entre la madurez tecnológica de los modelos de lenguaje y la infraestructura de hardware de los dispositivos móviles modernos crea las condiciones necesarias para desarrollar soluciones de voz robustas, seguras y completamente autónomas que no dependan de servicios externos.

1.2. Estado del arte

En la actualidad, los asistentes de voz de propósito general han alcanzado un grado de madurez notable en tareas de reconocimiento automático de habla y comprensión del lenguaje natural [6, 7]. Sin embargo, la adopción e integración de estas tecnologías en dominios de misión crítica y alta seguridad, como el sector bancario, sigue siendo limitada. Las soluciones bancarias que han incorporado interfaces conversacionales suelen basarse en integraciones en la nube, donde los flujos de audio y datos se envían a servidores externos para su procesamiento. Este enfoque tradicional genera legítimas preocupaciones respecto a la privacidad, la latencia y la exposición de información financiera sensible [8].

Entre los casos de referencia más relevantes a nivel internacional, se destaca Erica [9], el asistente virtual de Bank of America, lanzado en 2018 y considerado uno de los primeros asistentes financieros conversacionales en alcanzar escala masiva. Erica permite a los usuarios realizar consultas de saldo, revisar el historial de transacciones y recibir recomendaciones personalizadas mediante lenguaje natural. Sin embargo, su arquitectura depende del procesamiento en la nube, lo que implica la transmisión continua de datos financieros sensibles a servidores externos. De manera similar, iniciativas como Eno de Capital One [10] y los chatbots bancarios desplegados en diversas entidades europeas siguen el mismo paradigma de conectividad remota.

En el ecosistema de Apple, la integración de asistentes de voz en aplicaciones financieras ha explorado el uso de SiriKit [11], un framework que permite a los desarrolladores exponer ciertas funcionalidades de sus aplicaciones al asistente nativo del sistema operativo. No obstante, esta integración presenta limitaciones en cuanto a la personalización del flujo conversacional y a la capacidad de interpretar intenciones transaccionales complejas y específicas del dominio bancario. El control sobre el procesamiento del lenguaje natural recae sobre los servidores de Apple, lo que no garantiza el nivel de privacidad requerido para operaciones financieras sensibles.

Para mitigar estos riesgos, la tendencia tecnológica emergente apunta hacia el procesamiento en el dispositivo. Bajo este paradigma, los modelos de inteligencia artificial se ejecutan localmente utilizando el hardware del dispositivo móvil, lo que garantiza que los datos acústicos y las intenciones transaccionales del usuario nunca abandonen su teléfono. Esta arquitectura maximiza la seguridad y la confidencialidad, pilares fundamentales para la viabilidad de cualquier solución dentro de la industria financiera. El avance en la compresión de modelos de lenguaje, técnicas como la cuantización y la destilación de conocimiento, han hecho factible ejecutar modelos de comprensión del lenguaje natural de alta precisión

directamente sobre el procesador neuronal (*Neural Engine*) de los chips de la serie A y M de Apple [12, 13], lo que abrió una nueva generación de aplicaciones de IA privadas y sin latencia de red.

1.3. Objetivos y alcance

El objetivo principal de este trabajo es diseñar, desarrollar y evaluar un prototipo funcional de un asistente transaccional por voz integrado en el entorno de una aplicación móvil para iOS. Este prototipo servirá como una prueba de concepto destinada a validar la viabilidad técnica de un sistema que permita a los usuarios ejecutar operaciones financieras mediante comandos de voz en lenguaje natural.

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Mejorar la accesibilidad e inclusión: desarrollar un canal de interacción alternativo que empodere a personas con discapacidades visuales o motoras, que reduzca la dependencia de la navegación táctil tradicional.
- Garantizar la seguridad y privacidad de los datos: implementar modelos de machine learning optimizados para ejecutarse de manera estrictamente local mediante Core ML de Apple [14], asegurando que el audio y los datos sensibles del usuario se procesen íntegramente en el dispositivo.
- Optimizar la experiencia de usuario: simplificar operaciones bancarias complejas condensándolas en un único paso conversacional, y reduzca de esta manera la fricción y el tiempo necesario para completar una transacción.
- Sentar bases técnicas para futuras auditorías: generar la documentación arquitectónica y técnica necesaria para facilitar una futura evaluación y validación por parte del equipo de ciberseguridad de la entidad bancaria.

1.3.1. Alcance y limitaciones

Con el propósito de mantener el enfoque en la validación tecnológica del motor de inteligencia artificial y la experiencia de usuario, el trabajo se delimita bajo parámetros estrictos de alcance.

- Dentro del alcance: el desarrollo contempla exclusivamente la creación de una aplicación prototipo nativa para iOS, que operará de manera independiente y aislada. El núcleo técnico abarcará la implementación del pipeline de inteligencia artificial en el dispositivo físico, integrando la captura segura de audio, un modelo de reconocimiento automático de habla y un módulo de interpretación de lenguaje natural para extraer intenciones y entidades clave. Dado el carácter de la prueba de concepto, todo el flujo de transacción, la pantalla de confirmación y los resultados de las operaciones se simularán utilizando datos completamente ficticios.
- Fuera del alcance: el trabajo no contempla la integración del prototipo, ni de sus componentes, con el código fuente de la aplicación productiva oficial, ni la conexión con APIs, servicios de backend o bases de datos reales de la institución financiera. Asimismo, queda excluida la puesta en marcha en producción y la publicación en la App Store. Desde el punto de vista del manejo de la información, está estrictamente excluido el uso o acceso a

datos reales de clientes, así como la ejecución de transacciones que movi-
licen dinero real. Finalmente, el sistema se limitará al español rioplatense
focalizado, dejando fuera del alcance el soporte multilingüe y operaciones
financieras distintas a transferencias y consultas.

1.4. Motivación: accesibilidad e inclusión

El paradigma de diseño predominante en las aplicaciones de banca móvil actuales se basa casi exclusivamente en la interacción táctil y visual. Si bien este enfoque resulta eficiente para la mayoría de los usuarios, presenta barreras tecnológicas y de usabilidad significativas para grupos demográficos importantes. Por un lado, las personas con discapacidades o disminuciones visuales enfrentan el desafío de navegar a través de menús complejos, campos de texto y botones, tareas que a menudo resultan frustrantes incluso con el soporte de las herramientas de accesibilidad nativas de los sistemas operativos. Por otro lado, las personas con discapacidades motoras encuentran que la necesidad de una manipulación precisa de la pantalla dificulta o impide directamente el uso autónomo de la aplicación para realizar operaciones financieras cotidianas.

En Argentina, según datos del Instituto Nacional de Estadística y Censos (INDEC), aproximadamente el 10 % de la población presenta algún tipo de dificultad o limitación permanente [15]. Dentro de este universo, las limitaciones visuales y motoras representan una proporción significativa, lo que configura un segmento de usuarios que históricamente ha sido desatendido por el diseño centrado en la interfaz táctil. A nivel global, la Organización Mundial de la Salud (OMS) estima que más de 2.200 millones de personas presentan alguna forma de deficiencia visual, de las cuales una fracción importante se encuentra en edad económicamente activa y requiere acceso a servicios financieros [16]. La brecha entre la necesidad de acceso a servicios bancarios y la capacidad de las interfaces actuales para satisfacer dicha necesidad constituye un problema de equidad e inclusión que la tecnología tiene el potencial y la responsabilidad de resolver.

La motivación principal de este trabajo es derribar estas barreras tecnológicas, promoviendo una inclusión financiera real mediante el desarrollo de un canal de interacción alternativo. La implementación de un asistente transaccional por voz no solo genera un impacto social positivo al mejorar radicalmente la accesibilidad, sino que también ofrece una experiencia de usuario superior, simplificando operaciones complejas en un único paso conversacional y reduzca la fricción del proceso. Más aún, al garantizar que todo el procesamiento ocurra de forma local en el dispositivo, se elimina la resistencia que muchos usuarios podrían tener hacia el uso de comandos de voz en entornos de alta sensibilidad, como lo es la gestión de sus finanzas personales.

Capítulo 2

Introducción específica

El desarrollo de una solución basada en el procesamiento de lenguaje natural exige un acoplamiento profundo y eficiente con el hardware del dispositivo móvil. Por consiguiente, para la construcción de esta prueba de concepto se optó por prescindir de tecnologías multiplataforma, que adapta íntegramente el ecosistema de desarrollo nativo de Apple. Esta decisión arquitectónica resulta indispensable para garantizar un acceso de bajo nivel a los coprocesadores neuronales y a las interfaces seguras de captura de audio. Este apartado detalla el ecosistema tecnológico y de software seleccionado para el desarrollo del prototipo.

2.1. Marco metodológico

Para la concepción, desarrollo y evaluación del sistema de transacciones financieras por voz, se adoptó como marco de trabajo la metodología CRISP-DM (*Cross-Industry Standard Process for Data Mining*) [17]. Si bien CRISP-DM fue concebida originalmente para proyectos de minería de datos, su estructura iterativa y centrada en el ciclo de vida de los datos la convierte en el estándar de facto para la ingeniería de proyectos de inteligencia artificial y machine learning. La elección de esta metodología responde a la necesidad de contar con un enfoque estructurado que permita alinear los objetivos técnicos del procesamiento de lenguaje natural con los requisitos críticos y las restricciones de seguridad propios de una aplicación de banca móvil. El ciclo de vida se ha adaptado específicamente para abordar el desarrollo de un pipeline NLU (*Natural Language Understanding*) [18] de ejecución local dentro del ecosistema iOS.

2.2. Herramientas utilizadas

Con el objetivo de cumplir con los requerimientos de procesamiento local (*Edge AI*) [19] y garantizar la privacidad por diseño (*Privacy by Design*) [20] en el contexto de una aplicación financiera, se adoptó de manera integral el entorno nativo de Apple. A continuación, se enumeran las herramientas integradas en la arquitectura del sistema.

- Swift [21]: es un lenguaje de programación compilado, multiparadigma y de tipado estático fuerte, desarrollado por Apple para la creación de software en sus sistemas operativos.

Su adopción se justifica por su alto rendimiento computacional y su seguridad en el manejo de memoria mediante el Conteo Automático de Referencias (ARC). Estas características son esenciales para manipular flujos de

datos en tiempo real (como el audio) y procesar estructuras de datos complejas sin incurrir en fugas de memoria o vulnerabilidades durante la ejecución transaccional.

- Xcode [22]: es el Entorno de Desarrollo Integrado (IDE) oficial y nativo provisto por Apple, que incluye los compiladores, depuradores y simuladores necesarios para el ciclo de vida del software.

Se utilizó como la herramienta central para la codificación, compilación y empaquetado del SDK. Su uso es mandatorio para desplegar la prueba de concepto en dispositivos físicos y para utilizar la herramienta *Instruments*, la cual permitió perfilar y auditar el consumo de memoria y CPU de los modelos de Inteligencia Artificial durante la inferencia local.

- AVFoundation [23]: es el *framework* nativo de Apple que proporciona una interfaz de programación de bajo nivel para interactuar con el hardware audiovisual del dispositivo.

Su uso fue indispensable para orquestar la captura segura del comando de voz. Específicamente, se utilizó la clase `AVAudioSession` para gestionar los permisos de privacidad del micrófono a nivel del sistema operativo, y la clase `AVAudioEngine` para interceptar la onda sonora cruda y aislar el flujo acústico sin depender de librerías de terceros.

- Speech Framework [24]: es una Interfaz de Programación de Aplicaciones (API) de Apple dedicada exclusivamente al Reconocimiento Automático de Habla (ASR), capaz de transcribir señales de audio a cadenas de texto.

Se justificó su uso frente a otras alternativas comerciales debido a su capacidad de operar en modo *offline*. Al forzar la propiedad `requiresOnDeviceRecognition`, esta herramienta garantizó que la voz del usuario nunca fuera transmitida a servidores externos, mitigando los vectores de ataque y cumpliendo con el estándar de secreto bancario requerido por el proyecto.

- Core ML [25]: es el *framework* central de *Machine Learning* de Apple, diseñado para integrar, optimizar y ejecutar modelos predictivos directamente en el sistema operativo iOS. Constituye el motor analítico del proyecto.

Se seleccionó porque delega automáticamente las cargas matemáticas pesadas (como la clasificación de intenciones y la extracción de entidades) al *Apple Neural Engine* (ANE). Esto garantiza que los algoritmos de Procesamiento de Lenguaje Natural (PLN) se ejecuten con una latencia mínima y un impacto energético marginal, habilitando el paradigma de inferencia 100 % *On-Device*.

- Coremltools [26]: es un paquete de código abierto basado en Python, desarrollado por Apple, que unifica la conversión de modelos entrenados en librerías estándar (como PyTorch o TensorFlow) al formato propietario `.mlpackage`.

Su aplicación metodológica fue crítica para viabilizar el despliegue de las redes neuronales en el entorno móvil. Se utilizó para aplicar técnicas de cuantización a los pesos sinápticos de los modelos, reduciendo drásticamente su tamaño de almacenamiento y la huella en la memoria RAM del dispositivo, sin comprometer significativamente la exactitud predictiva del sistema.

2.3. Requerimientos y modelos de IA utilizados

El núcleo algorítmico del prototipo aborda la comprensión del comando de voz como un problema predictivo dual de machine learning: la clasificación de intenciones y el etiquetado de entidades transaccionales.

Por un lado, la clasificación multiclase a nivel de oración tiene como objetivo inferir la acción principal del usuario (por ejemplo, iniciar una transferencia o consultar un saldo). Matemáticamente, la red neuronal proyecta la secuencia de texto vectorizada hacia un espacio discreto de clases predefinidas, empleando una función de activación *softmax* para seleccionar, de manera excluyente, la intención que maximiza la probabilidad estadística.

Por otro lado, la conformación del *payload* financiero exige resolver un problema de etiquetado de secuencias a nivel de *token*, enmarcado en el Reconocimiento de Entidades Nombradas (NER)[27]. En lugar de emitir un único juicio para toda la oración, el modelo asigna una clase categórica a cada palabra utilizando el esquema de anotación IOB (*Inside*, *Outside*, *Beginning*). Esta técnica permite al sistema demarcar los límites exactos de variables críticas, como el monto numérico, la divisa y el destinatario, capturando las dependencias semánticas del lenguaje para extraer los parámetros exactos de la operación.

2.4. Exploración teórica de arquitecturas evaluadas

Para abordar este desafío predictivo dual y que se respeten las restricciones de memoria y procesamiento de los dispositivos móviles, se llevó a cabo una evaluación teórica de diversas arquitecturas algorítmicas, que analiza el compromiso (trade-off) entre precisión semántica, latencia de inferencia y consumo de recursos.

En primer lugar, se analizó el estado del arte representado por las arquitecturas basadas en Transformers. Este paradigma, fundamentado en el mecanismo de autoatención, prescinde de la recurrencia secuencial y permite capturar dependencias a largo plazo y procesa el contexto de manera bidireccional y paralelizable. Dentro de esta familia, se evaluó la viabilidad de incorporar modelos del lenguaje preentrenados adaptados al español, como BETO (la variante hispanohablante de BERT) [28]. Si bien BETO ofrece un rendimiento superior en la comprensión de las sutilezas léxicas y morfológicas del lenguaje coloquial rioplatense, su implementación nativa en el edge presenta obstáculos significativos. La alta dimensionalidad de sus parámetros (típicamente superior a 110 millones) exige la aplicación de técnicas agresivas de compresión, poda (*pruning*) y destilación de conocimiento (knowledge distillation) para transformarlo en un artefacto viable para el entorno iOS sin agotar la memoria RAM del dispositivo.

Como alternativa de menor huella computacional, se exploró la familia de las redes neuronales recurrentes (RNN) [29]. A diferencia de los transformers, las RNN procesan los tokens de manera estrictamente secuencial, actualiza un estado oculto que actúa como memoria temporal del contexto lingüístico. Dado que las arquitecturas RNN tradicionales son susceptibles al problema del desvanecimiento del gradiente (*vanishing gradient*) al procesar secuencias largas, la investigación se centró en sus variantes con mecanismos de compuerta (*gating*): las redes de memoria a corto y largo plazo (LSTM) [30] y las unidades recurrentes

con compuerta (GRU) [31]. Estas topologías regulan matemáticamente el flujo de información, dice qué características del contexto previo deben retenerse y cuáles descartarse. Históricamente, las arquitecturas basadas en LSTM bidireccionales (BiLSTM) [32] han demostrado una alta eficacia para el etiquetado de secuencias (NER) [27], logra una precisión competitiva con un costo computacional y tamaño de modelo órdenes de magnitud inferiores a los de un modelo fundacional, lo que las hace inherentemente más compatibles con inferencias On-Device [33].

Finalmente, se evaluó la pertinencia de utilizar los frameworks de comprensión de lenguaje natural nativos de Apple, en particular la API NaturalLanguage y el ecosistema de Create ML [34]. Apple proporciona clases altamente especializadas, como NLMModel y NLTagger, diseñadas para compilar clasificadores de texto y extractores de entidades a partir de *datasets* etiquetados. La ventaja superlativa de este enfoque reside en su integración opaca y de bajo nivel con el silicio del dispositivo. Los modelos generados bajo este estándar están nativamente optimizados para aprovechar la aceleración por hardware del procesamiento neural de Apple, garantiza tiempos de latencia del orden de los milisegundos y un impacto insignificante en la autonomía de la batería. Aunque la utilización de frameworks cerrados sacrifica el grado de control arquitectónico y la hiperparametrización profunda que ofrecen librerías como PyTorch o TensorFlow, su robustez operativa, la garantía de privacidad al aislar el procesamiento de la red, y la facilidad de integración mediante Core ML, posicionan a estas herramientas nativas como candidatos estratégicos para validar la viabilidad técnica de una prueba de concepto en el sector financiero.

Capítulo 3

Implementación y desarrollo

Este capítulo expone el proceso técnico de implementación y desarrollo del sistema propuesto. En primer lugar, se describe la arquitectura de la solución, instancia en la cual se definió la separación de responsabilidades entre la aplicación base y el SDK nativo denominado Aurum. A continuación, se detalla el modelo de datos estructurado que se diseñó para encapsular las intenciones y entidades extraídas de los comandos de voz. Finalmente, se explican los componentes del código fuente del prototipo.

3.1. Arquitectura del sistema

A continuación, se detalla la instanciación de cada una de las seis fases de la metodología al contexto particular de esta prueba de concepto.

- **Comprensión del negocio:** esta fase inicial constituye el pilar fundamental del trabajo, dado que define el puente entre la necesidad de accesibilidad del usuario y las exigencias transaccionales del sistema bancario. El problema central del negocio radica en las barreras de usabilidad que las interfaces táctiles tradicionales imponen a personas con discapacidades visuales o motoras. La solución propuesta es habilitar un canal conversacional.

Desde la perspectiva del sistema, la comprensión del negocio se traduce en el desafío algorítmico de transformar lenguaje natural, inherentemente ambiguo y desestructurado, en formatos estructurados y accionables que el core bancario pueda procesar de manera segura. Por ejemplo, una expresión coloquial como "transferile cinco mil pesos a mamá" carece de utilidad directa para una base de datos financiera. El objetivo del negocio exige que el pipeline de IA sea capaz de capturar esta emisión acústica y extraer inequívocamente una intención principal (Clasificación de Intención: TRANSFERENCIA) y los parámetros o entidades asociadas (Reconocimiento de Entidades Nombradas: MONTO: 5000, MONEDA: ARS, DESTINATARIO: mamá). Solamente al mapear el habla hacia este diccionario de datos estructurado (típicamente en formato JSON o estructuras nativas de Swift), el sistema móvil puede invocar la lógica de negocio correspondiente, lo que garantiza una operación determinista, libre de ambigüedades y sin fricción para el usuario.

- **Comprensión de los datos:** una vez definido el objetivo de extraer intenciones y entidades transaccionales, esta fase se centra en identificar la naturaleza y los requerimientos de la información necesaria para entrenar el

modelo. En el contexto de la banca móvil, los datos reales de los usuarios están protegidos por estrictas normativas de privacidad y secreto bancario. Al tratarse de una prueba de concepto y ante la ausencia de un registro histórico de comandos de voz accionados por usuarios, se determinó la necesidad de construir un conjunto de datos (*dataset*) sintético. En esta etapa se analizó la morfología del lenguaje coloquial rioplatense aplicado a finanzas, identifica sinónimos de acciones (transferir, enviar, pasar plata), variaciones en la expresión de montos (números cardinales, fracciones) y formas comunes de nombrar contactos.

- **Evaluación:** esta fase contrastó los resultados empíricos del modelo contra los criterios de éxito definidos al inicio del trabajo. A nivel algorítmico, se midió el desempeño predictivo que utiliza un conjunto de prueba aislado, que calcula métricas de Accuracy para la clasificación de intenciones, así como Precision, Recall y F1-Score para el reconocimiento de entidades. A nivel de negocio e integración, la evaluación trascendió las métricas puramente matemáticas para incorporar pruebas de usabilidad en el entorno simulado de iOS, mide la latencia de la inferencia local y la tasa de éxito de la tarea integral. Esto permitió verificar que la extracción estructurada lograra, en efecto, completar el flujo de la aplicación sin errores perjudiciales para la experiencia del usuario.
- **Despliegue:** en un proyecto de analítica tradicional, el despliegue suele implicar la exposición del modelo a través de una API en un servidor en la nube. Sin embargo, en esta prueba de concepto, la fase de despliegue se redefinió para cumplir con el requisito innegociable de máxima privacidad. El despliegue consistió en la conversión de los modelos entrenados al formato `.mlpackage`, su integración directa en el código fuente del proyecto en Xcode, y su invocación local mediante el framework Core ML. De esta manera, el modelo se "desplegó" encapsulado dentro del propio binario de la aplicación iOS, se garantiza que el pipeline de transformación de voz a texto (vía AVFoundation y Speech) y la posterior comprensión del lenguaje natural se ejecuten de manera estrictamente local en el dispositivo del usuario final.

El diseño arquitectónico de la presente prueba de concepto se fundamenta en el paradigma de computación Edge AI, trasladando la carga computacional de los algoritmos de inteligencia artificial directamente al dispositivo móvil del usuario. Esta decisión estratégica persigue un doble objetivo metodológico y normativo: por un lado, minimizar la latencia inherente a las comunicaciones de red corporativas y, por otro, garantizar un entorno de privacidad por diseño. Al ejecutar los modelos a través del *framework* Core ML, los flujos de audio y las transcripciones de índole financiera no abandonan en ningún momento el entorno físico del hardware de Apple, lo que mitiga drásticamente los vectores de ataque asociados a la interceptación de datos en tránsito o al almacenamiento temporal en servidores de terceros. Para asegurar la escalabilidad, la modularidad y la mantenibilidad del código, el sistema fue concebido bajo el principio de separación de responsabilidades. La solución se estructura en dos grandes bloques funcionales e independientes: la aplicación base anfitriona y el SDK nativo Aurum [35]. Es imperativo establecer una restricción arquitectónica crítica que define los límites de esta integración: el proyecto no incluye, bajo ninguna circunstancia, algoritmos de validación biométrica de voz. La autenticación de la identidad del cliente,

el manejo de tokens de sesión y las medidas de seguridad perimetral recaen, de manera exclusiva y excluyente, sobre la aplicación móvil del banco. En este modelo topológico, el SDK Aurum opera bajo la premisa de confianza de que es invocado dentro de un contexto transaccional previamente securizado por la entidad financiera, lo que hace adoptar el dominio de responsabilidad estrictamente a la captura acústica, la transcripción y la extracción semántica de intenciones y entidades.

En la figura 3.1 se presenta el diagrama de componentes.

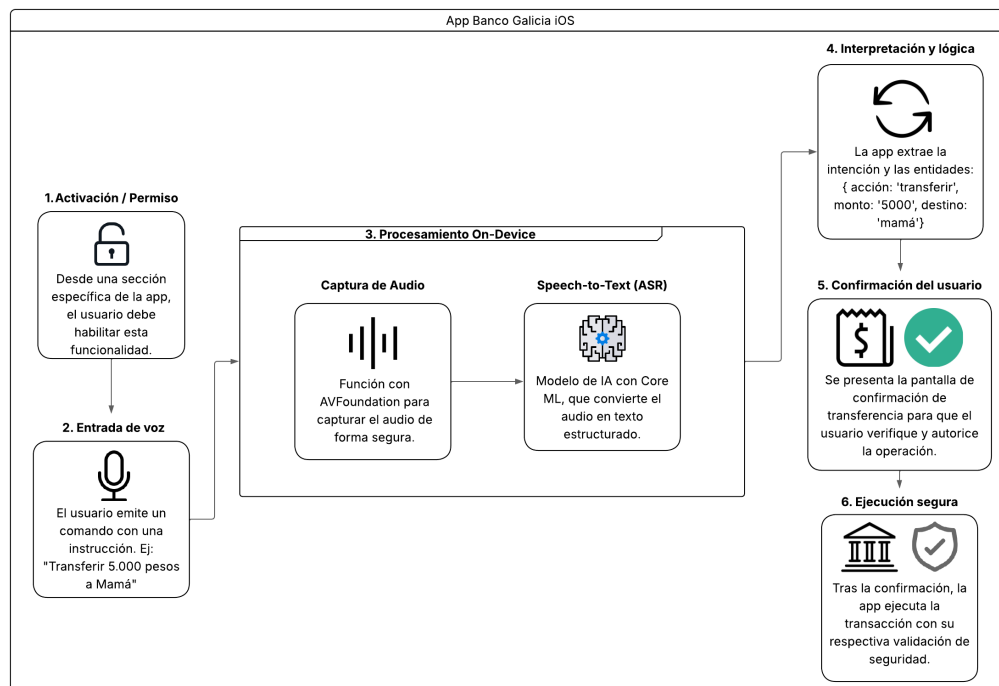


FIGURA 3.1. Diagrama de arquitectura de componentes que conforman la prueba de concepto.

El ciclo de vida de una transacción impulsada por voz atraviesa un pipeline determinista que fluye secuencialmente a través de las capas de la arquitectura descrita. El proceso de ingesta de datos se desencadena en la aplicación base cuando el usuario, luego de superar los controles de acceso convencionales del banco, interactúa con el disparador de la interfaz gráfica (botón de dictado). En ese instante, la aplicación anfitriona invoca la API pública del SDK Aurum, transfiriéndole el control del hilo de ejecución de audio. El SDK inicializa una sesión de captura de alto rendimiento mediante el framework AVFoundation, configurando el entorno acústico adecuado para aislar la frecuencia de la voz humana. Una vez digitalizada la señal de audio, el motor interno del SDK emplea las capacidades de transcripción fuera de línea (offline) provistas por las librerías nativas de iOS para ejecutar el reconocimiento automático de habla, convirtiendo el espectrograma en una cadena de texto cruda. Acto seguido, este texto es sometido a técnicas de normalización morfológica y es inyectado en el motor de comprensión del lenguaje natural. En esta fase de inferencia, la transcripción es procesada localmente por los modelos vectoriales preentrenados y cuantizados alojados en Core ML. Dichos modelos resuelven algorítmicamente el problema de clasificación de la intención del usuario y proceden al etiquetado posicional para extraer

las variables nominales del comando. El flujo de información concluye cuando la salida matricial de las redes neuronales es traducida por el SDK en un payload [36] de datos estructurado y fuertemente tipado. Este artefacto resultante, que encapsula inequívocamente la acción solicitada, la magnitud numérica del monto, el tipo de divisa y la identidad del beneficiario, es retornado de manera asíncrona a la aplicación anfitriona a través de patrones de delegación (*Delegates o Closures*)[21]. A partir de la entrega de este objeto estructurado, el SDK Aurum finaliza su intervención y cede nuevamente el control a la aplicación base que asume la responsabilidad de mapear el payload contra su modelo de datos transaccional, renderizar la interfaz de validación final ante el usuario y proceder con la comunicación cifrada hacia el core bancario.

En la figura 3.2 se muestra el flujo del pipeline (ciclo de vida de una transacción por voz).

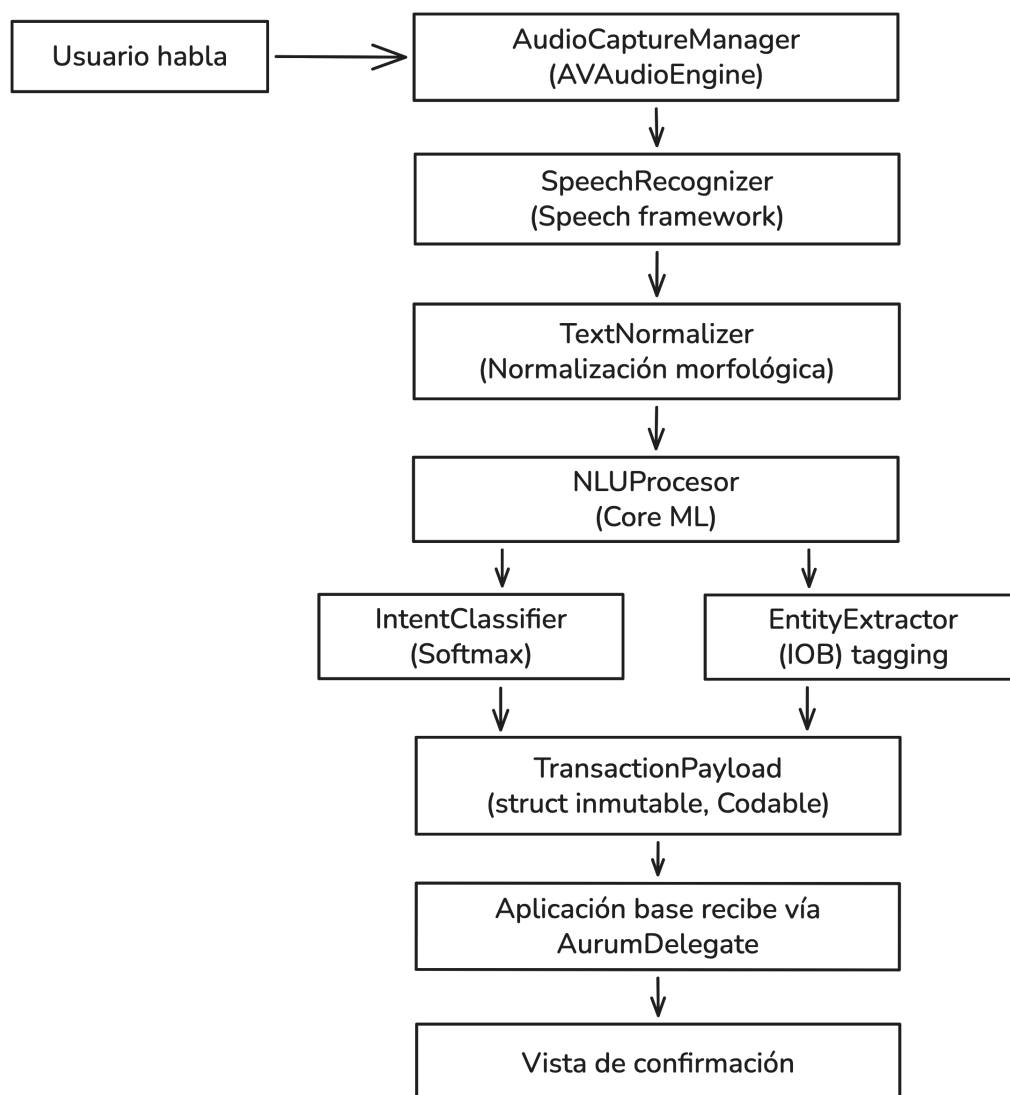


FIGURA 3.2. Flujo de pipeline de una transacción realizada por voz.

3.2. Tratamiento y preparación de datos

El desarrollo de modelos de procesamiento de lenguaje natural aplicados a dominios de misión crítica, como el sector bancario, enfrenta el desafío inherente del acceso a la información, conocido como el problema de “arranque en frío” (*cold start*). Las estrictas normativas de privacidad y el secreto bancario imposibilitan el uso de corpus conversacionales de clientes reales sin someterlos previamente a procesos de anonimización complejos y costosos. Adicionalmente, al tratarse de una prueba de concepto que introduce un canal de interacción inédito en el entorno simulado de la aplicación móvil, no existe telemetría histórica ni registros acústicos o textuales previos de comandos transaccionales. Ante esta restricción, se determinó como exigencia metodológica la generación de un conjunto de datos sintéticos (*mock data*). La construcción de un corpus artificial permite no solo superar la ausencia de datos iniciales, sino también ejercer un control riguroso sobre la distribución probabilística de las clases (intenciones) y garantizar una cobertura exhaustiva de los casos límite (*edge cases*) de las entidades. Este enfoque metodológico aísla el desarrollo algorítmico de los riesgos de privacidad y provee un entorno determinista, fundamental para iterar la arquitectura de la red neuronal durante las primeras fases de evaluación.

3.2.1. Creación de plantillas de comandos y aumentación de datos

La síntesis del corpus de entrenamiento se estructuró a partir de la definición de plantillas (*templates*) gramaticales. Estas plantillas modelan lógicamente las posibles estructuras sintácticas que un usuario emplearía para interactuar con el sistema bancario y se abstraen los valores concretos, tales como: [ACCIÓN] [MONTO] [MONEDA] a [DESTINATARIO]. Una vez consolidadas las estructuras base, se implementaron técnicas de aumentación de datos (*data augmentation*) para inyectar la variabilidad inherente al lenguaje natural [37].

La transición entre el entorno probabilístico inherente a los modelos de machine learning y el rigor determinista exigido por los sistemas financieros requiere de una capa de abstracción robusta. Para resolver este paradigma, se estructuraron los datos resultantes de la inferencia algorítmica utilizando estructuras de valor *struct* en Swift [21]. El uso de *structs* garantizó la inmutabilidad de la información una vez extraída y procesada, previniendo alteraciones accidentales de las variables en la memoria durante el ciclo de vida de la transacción. Para la serialización de este modelo, se adoptó el protocolo Codable nativo de Apple [38]. Esta decisión arquitectónica se justificó en la necesidad de estandarizar la comunicación entre el SDK y la aplicación base, permitiendo que el objeto Swift generado en memoria pudiera ser codificado y decodificado de manera nativa hacia formatos universales de intercambio de datos, como JSON [39], sin depender de librerías de terceros.

3.2.2. Modelo de datos

El diseño del payload (o carga útil) que el modelo de etiquetado de secuencias (NER) y el clasificador de intenciones devolvieron al sistema se consolidó en una única entidad semántica. Durante el desarrollo, se definieron campos fuertemente tipados para encapsular de manera exacta las variables transaccionales críticas:

- **Intención:** representa la acción directiva inferida del comando verbal (por ejemplo, iniciar una transferencia). Se estructuró como el eje central que define el ruteo posterior dentro de la aplicación.
- **Monto:** se tipificó mediante el uso de datos de punto flotante de doble precisión (*Double*) [21], permitiendo la representación matemática de la magnitud monetaria extraída de la transcripción fonética.
- **Moneda:** se modeló utilizando enumeraciones (*Enum*) con tipos crudos de cadena de texto (*String raw values*) [21]. Esto restringió el dominio de la divisa a valores finitos y normalizados bajo estándares internacionales (ej. ARS o USD), eliminando la ambigüedad de sinónimos coloquiales capturados por el motor NLU, como "pesos", "lucas" o "dólares".
- **Destinatario:** se definió como una cadena de caracteres cruda (*String*) [21] que encapsula la identidad nominal del beneficiario de la operación, dejando el valor preparado para ser contrastado posteriormente contra la agenda de contactos o base de alias validada del banco.

Este enfoque de modelado determinista cumple una función vital en la arquitectura de integración del sistema. Al transformar una emisión acústica originalmente desestructurada en un objeto Swift predecible y validado a nivel de compilador, se abstraigo a la aplicación anfitriona de toda la complejidad inherente al procesamiento de lenguaje natural. En consecuencia, la integración con los servicios de backend del banco se facilita de manera significativa: la aplicación móvil anfitriona consume un payload limpio, cuya estructura es análoga a la que generaría una interfaz táctil tradicional basada en formularios. Esto habilita a la aplicación base a tomar dichas variables tipadas e inyectarlas de manera directa en los constructores de sus peticiones de red seguras, manteniendo intacta la lógica de consumo de los endpoints transaccionales de la institución.

3.3. Desarrollo del SDK (Aurum)

El desarrollo de Aurum se abordó bajo el paradigma de la programación orientada a componentes, se materilizó como un *framework* dinámico y reutilizable dentro del ecosistema de Apple. El propósito fundamental de este encapsulamiento radicó en abstraer la alta complejidad algorítmica inherente al procesamiento de señales acústicas y a la inferencia neuronal, proveyendo a la aplicación bancaria anfitriona de una "caja negra" funcional. Al aislar el pipeline de inteligencia artificial en un módulo independiente, se garantizó la cohesión del código y se facilitó su futura distribución y mantenimiento, sin contaminar la lógica de negocio ni la interfaz de usuario de la aplicación base. Para orquestar la ejecución secuencial de las tareas de procesamiento, se implementó una clase principal o *facade* (fachada) dentro del SDK, responsable de coordinar el ciclo de vida integral de la transacción de voz. El primer eslabón de esta coordinación consistió en la gestión del hardware de audio del dispositivo. A través de la API AVAudioSession, se configuró el entorno acústico solicitando los permisos de grabación al sistema operativo y estableciendo la categoría de la sesión para priorizar la entrada de voz (record o playAndRecord). Seguidamente, se instanció un AVAudioEngine para capturar el flujo continuo de muestras de audio provenientes del micrófono y canalizarlas hacia los nodos de procesamiento. Una vez establecido el buffer de audio, la clase principal integró el *framework* nativo Speech [24] para

ejecutar la transcripción de voz a texto. Para cumplir con las estrictas normativas de privacidad del sector financiero, se forzó el procesamiento estrictamente local configurando la propiedad `requiresOnDeviceRecognition` en verdadero (`true`). Esta restricción arquitectónica impidió que los paquetes de audio fueran transmitidos a los servidores en la nube de Apple. Posteriormente, la cadena de texto transcrita fue inyectada en el módulo de comprensión de lenguaje natural. En esta instancia, el SDK invocó los modelos de machine learning, previamente entrenados y cuantizados, a través del *framework* Core ML. La ejecución local de estos modelos permitió clasificar la intención transaccional y extraer las entidades relevantes (monto, moneda, destinatario) con una latencia del orden de los milisegundos, aprovechando la aceleración por hardware del dispositivo. Para consolidar la integración, el SDK Aurum fue dotado de una Interfaz de Programación de Aplicaciones (API) pública minimalista [40]. Dado que la captura de audio y la inferencia neuronal son procesos no bloqueantes que ocurren a lo largo del tiempo, la comunicación con la aplicación anfitriona se resolvió utilizando patrones asíncronos nativos de Swift, combinando el uso de delegados (*Delegates*) y clausuras (*Closures*). De este modo, la aplicación base únicamente necesita invocar un método público de inicio de escucha (por ejemplo, `startListening()`) y suscribirse para recibir la respuesta. Al finalizar el procesamiento, el SDK ejecuta un callback que retorna el payload de datos estructurado (detallado en la sección anterior) o, en su defecto, un error tipificado. Esta arquitectura basada en eventos asegura que la aplicación consumidora permanezca completamente agnóstica respecto a las operaciones matemáticas de vectorización, etiquetado y transcripción que ocurren bajo la superficie.

En la figura 3.3 se presenta el diagrama de flujo de los actores principales: la aplicación base (la aplicación del banco) y el SDK Aurum.

3.4. Desarrollo de la aplicación base

Para evaluar la viabilidad de la integración y el correcto funcionamiento del SDK Aurum, se desarrolló una aplicación móvil de prueba que emula el entorno transaccional de una entidad bancaria. Esta aplicación base se construyó con el propósito de actuar como el "contenedor cliente" que orquesta la experiencia del usuario y consume la API expuesta por el *framework* de procesamiento de lenguaje natural. El diseño de la interfaz de usuario se estructuró bajo principios de minimalismo y accesibilidad, focalizándose en el flujo de interacción conversacional. En la vista principal, se implementó un botón de activación por voz que actúa como el disparador (*trigger*) del ciclo transaccional. Para proveer una experiencia de usuario coherente y mitigar la incertidumbre del sistema, se diseñó un mecanismo de retroalimentación visual. Durante el estado de escucha, la interfaz modifica dinámicamente su estado gráfico para indicarle al usuario de forma inequívoca que el hardware de audio se encuentra activo, capturando y procesando el comando verbal en tiempo real. Una vez que el SDK finaliza la inferencia neuronal y la extracción de parámetros, la aplicación anfitriona recibe el payload estructurado de manera asíncrona a través de sus métodos delegados. Este intercambio determinista de datos permite a la aplicación base extraer las variables transaccionales (intención, monto, moneda y destinatario) e inyectarlas directamente en su modelo de vistas. Como resultado de este mapeo, la aplicación renderiza una pantalla

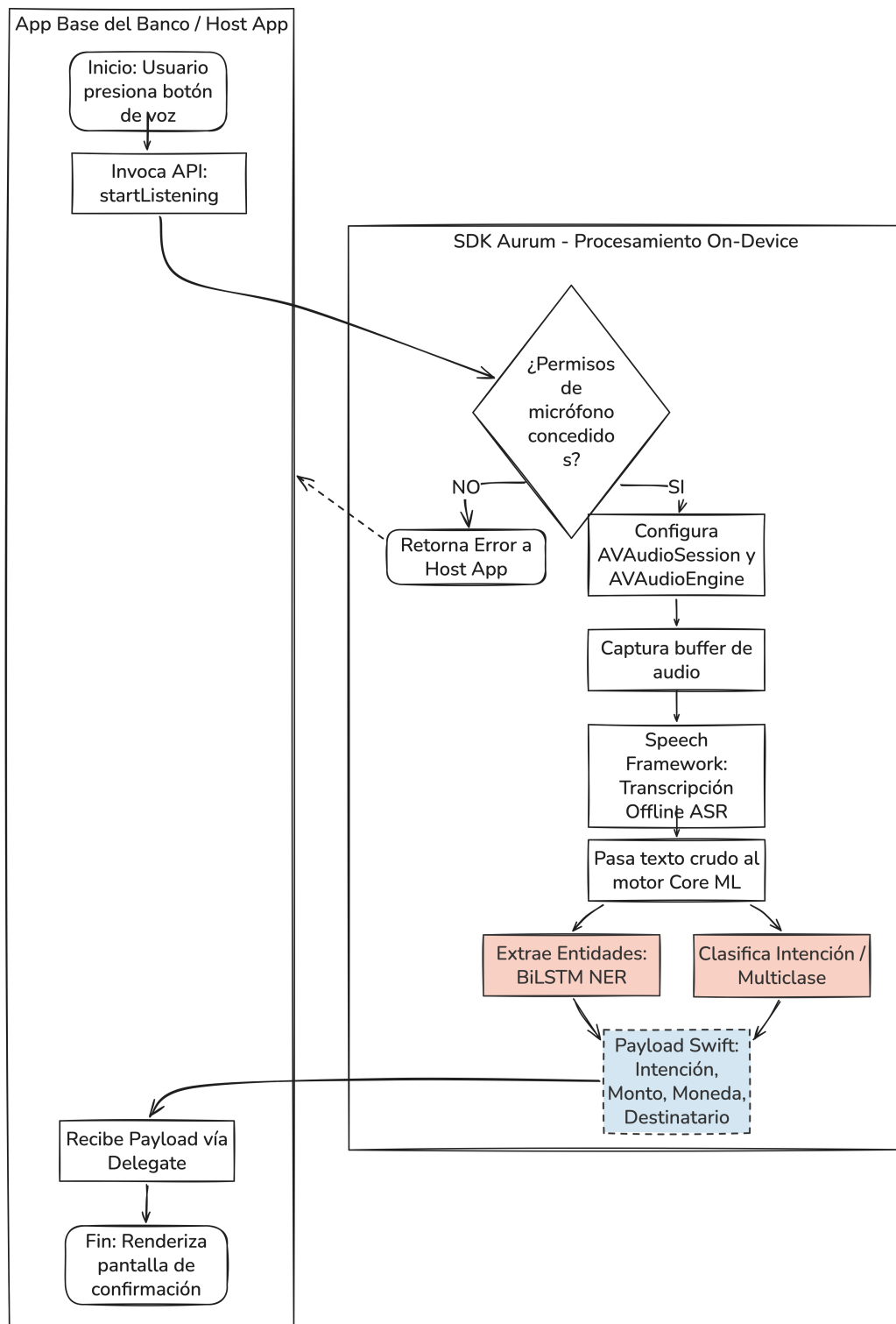


FIGURA 3.3. Diagrama de flujo de Aurum.

de confirmación simulada que expone la información procesada de manera legible y estructurada, brindando al usuario la oportunidad de auditar los datos extraídos de su propia voz antes de autorizar la ejecución de la transferencia.

Finalmente, resulta imperativo reiterar la restricción arquitectónica que rige el diseño de este sistema. En el contexto de esta prueba de concepto, la transacción concluye tras la confirmación en la interfaz gráfica simulada. No obstante, en un entorno productivo real, la aplicación base constituye el único perímetro de seguridad autorizado. Es estrictamente en esta capa superior de la aplicación base donde la entidad financiera ejecutaría sus métodos propietarios de validación de identidad, evaluación de riesgos y autenticación multifactor, tales como la validación biométrica facial o el ingreso de credenciales, de manera previa a la confirmación definitiva de la transacción, asegurando que el SDK Aurum opere exclusivamente como un facilitador de accesibilidad y no como un validador criptográfico.

Bibliografía

- [1] Ángeles Núñez y Laura Núñez Letamendia. «La transformación digital de la banca: innovación, colaboración y sostenibilidad.» En: *Harvard Deusto* (2024).
- [2] Informe económico y financiero. «El momento de la inteligencia artificial». En: *ESADE* (2023).
- [3] Leticia Rengifo. *Inclusión financiera en Argentina y el efecto Pandemia*. Visitado el 2026-03-25. 2022. URL: <https://repositorio.utdt.edu/items/c8641051-5953-4145-a6ce-f98147aa7093>.
- [4] Banco Central de la República Argentina. *Informe de Inclusión Financiera - Segundo semestre 2022*. <https://www.bcra.gob.ar/publicaciones/informe-de-inclusion-financiera-segundo-semestre-2022/>. Dic. de 2022. (Visitado 02-05-2023).
- [5] Diario Clarín. *El Galicia apuesta a seguir siendo el mayor banco privado de la Argentina*. Visitado el 2026-03-28. 2024. URL: https://www.clarin.com/80-aniversario/bancos-finanzas/galicia-apuesta-seguir-mayor-banco-privado-argentina_0_iVSN8H9vt1.html.
- [6] Frontline VC. *Voice AI and Financial Services: Introducing Avallon*. Visitado el 2026-03-29. 2025. URL: <https://frontline.vc/blog/voice-ai-and-financial-services-introducing-avallon/>.
- [7] Deepgram. *What Is a Voice AI Agent? The Complete Guide for 2026*. Visitado el 2026-03-29. 2026. URL: <https://deepgram.com/learn/what-is-a-voice-ai-agent-2026>.
- [8] Kardome. *The Voice Privacy Problem: Security and Latency in Cloud-Based Voice Assistants*. Visitado el 2026-03-29. 2025. URL: <https://www.kardome.com/resources/blog/voice-privacy-concerns/>.
- [9] Bank of America. *A Decade of AI Innovation: BofA's Virtual Assistant Erica Surpasses 3 Billion Client Interactions*. Visitado el 2026-03-29. 2025. URL: <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2025/08/a-decade-of-ai-innovation--bofa-s-virtual-assistant-erica-surpas.html>.
- [10] Amazon Web Services. *Capital One Completes Migration from Data Centers to AWS*. Visitado el 2026-03-29. 2023. URL: <https://aws.amazon.com/solutions/case-studies/capital-one-all-in-on-aws/>.
- [11] Apple Inc. *SiriKit: Payments Domain*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/sirikit/payments>.
- [12] Apple Inc. «Apple Intelligence Foundation Language Models». En: *arXiv preprint arXiv:2407.21075* (2024). URL: <https://arxiv.org/abs/2407.21075>.
- [13] Apple Machine Learning Research. *Deploying Transformers on the Apple Neural Engine*. Visitado el 2026-03-29. 2022. URL: <https://machinelearning.apple.com/research/neural-engine-transformers>.

- [14] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/coreml>.
- [15] Instituto Nacional de Estadística y Censos. *Estudio Nacional sobre el Perfil de las Personas con Discapacidad (EPPD): Resultados definitivos*. Inf. téc. Visitado el 2026-03-29. INDEC, República Argentina, 2018. URL: <https://www.indec.gov.ar/indec/web/Nivel4-Tema-2-41-135>.
- [16] Organización Mundial de la Salud. *Informe mundial sobre la visión*. Inf. téc. Visitado el 2026-03-29. World Health Organization, 2019. URL: <https://www.who.int/es/publications/i/item/9789241516570>.
- [17] Rüdiger Wirth y Jochen Hipp. «CRISP-DM: Towards a standard process model for data mining». En: *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*. Springer-Verlag. 2000, págs. 29-39.
- [18] Qian Chen, Zhu Zhuo y Wen Wang. «BERT for joint intent prediction and slot filling». En: *arXiv preprint arXiv:1902.10909* (2019).
- [19] Weisong Shi et al. «Edge Computing: Vision and Challenges». En: *IEEE Internet of Things Journal* 3.5 (2016), págs. 637-646.
- [20] Ann Cavoukian. «Privacy by Design: The 7 Foundational Principles». En: *Information and Privacy Commissioner of Ontario, Canada* 5 (2009), págs. 1-12.
- [21] Apple Inc. *The Swift Programming Language: Language Guide*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://docs.swift.org/swift-book/>.
- [22] Apple Inc. *Xcode: Integrated Development Environment*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/xcode/>.
- [23] Apple Inc. *AVFoundation: Work with audiovisual assets, control device cameras, process audio, and configure system audio interactions*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/avfoundation>.
- [24] Apple Inc. *Speech: Perform speech recognition on recorded or live audio*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/speech>.
- [25] Apple Inc. *Core ML: Integrate machine learning models into your app*. Visitado el 2026-04-02. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/coreml>.
- [26] Apple Inc. *coremltools: Unified conversion API and optimization tools for Core ML*. Visitado el 2026-04-12. Apple Open Source. 2026. URL: <https://coremltools.readme.io/>.
- [27] Guillaume Lample et al. «Neural Architectures for Named Entity Recognition». En: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, págs. 260-270.
- [28] José Cañete et al. «Spanish pre-trained bert model and evaluation data». En: *PML4DC at ICLR 2020*. 2020.
- [29] Jeffrey L Elman. «Finding structure in time». En: *Cognitive science* 14.2 (1990), págs. 179-211.
- [30] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.

- [31] Kyunghyun Cho et al. «Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation». En: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, págs. 1724-1734.
- [32] Zhiheng Huang, Wei Xu y Kai Yu. «Bidirectional LSTM-CRF models for sequence tagging». En: *arXiv preprint arXiv:1508.01991* (2015).
- [33] Saurabh Dhar et al. «A survey of on-device machine learning: An algorithms and learning theory perspective». En: *ACM Transactions on Internet of Things (TIOT)* 2.3 (2021), págs. 1-49.
- [34] Apple Inc. *Natural Language: Analyze natural language text and deduce its language-specific metadata*. Visitado el 2026-03-29. Apple Developer Documentation. 2025. URL: <https://developer.apple.com/documentation/naturallanguage>.
- [35] Apple Inc. *Creating a Framework: Package dynamic libraries and resources to share code*. Visitado el 2026-04-03. Apple Developer Documentation. 2026. URL: <https://developer.apple.com/documentation/xcode/creating-a-framework>.
- [36] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015.
- [37] Steven Y Feng et al. «A survey of data augmentation approaches for NLP». En: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021* (2021), págs. 968-988.
- [38] Apple Inc. *Encoding and Decoding Custom Types*. Visitado el 2026-04-12. Apple Developer Documentation. 2026. URL: https://developer.apple.com/documentation/foundation/archives_and_serialization/encoding_and_decoding_custom_types.
- [39] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. IETF, 2017. URL: <https://datatracker.ietf.org/doc/html/rfc8259>.
- [40] IEEE. «IEEE Standard Glossary of Software Engineering Terminology». En: *IEEE Std 610.12-1990* (1990), págs. 1-84.